

CDI Scorer — Documentation

Integrity Engineering

Building the Medical Record Validator (CDI Scorer) with Claude Code, end to end from an interactive Claude Code session — no Agent SDK script, no LLM at scoring time.

BUILD GUIDE

Medical Record Validator ("CDI Scorer") — Flask + React, rule-based rubric scoring

CLAUDE CODE | INTERACTIVE SESSION | INTERNAL ENGINEERING DOCS — REPRODUCTION OF THE ACTUAL BUILD SESSION

Contents

Unit 1	Disclaimer	3
Unit 2	Objective	3
Unit 3	Prerequisites	3
Unit 4	Source Code Organization	4
Unit 5	Summary of Steps Performed in this Session	5
Unit 6	Detailed Steps 0–8 (Step-by-Step, Real Artifacts)	6
Unit 7	Understand the Project Structure and Code	15
Unit 8	Step-by-Step Procedure to Rebuild	16

LAB: Medical Record Validator (CDI Scorer)

Disclaimer

This document reconstructs the **actual** Define → Build → Validate workflow used to build the CDI Scorer in a real Claude Code session — the real `SCOPE.md`/`SPEC.md`/`CLAUDE.md` content, the real code, and real captured terminal output are quoted throughout, not illustrative placeholders. Every rubric dimension's keyword/regex indicators were authored by Claude Code as a deterministic matching layer — there is **no LLM call at scoring time** and no clinical judgment; this is a demonstration app and must not be used for real clinical or billing decisions.

Objective

Provide a reproducible record of how an AI-assisted clinical-documentation review application was actually built with Claude Code: upload a medical record (PDF / TXT / DOCX), extract its text, score it against the fixed 10-dimension, weighted, narrative `med_record_rubrics.json` rubric using purely deterministic keyword/regex matching (no LLM judgment), and return a structured score / decision-band / gaps report through a Flask backend and a React front end.

Prerequisites

- Python 3.11+ (this build session ran on Python 3.14.7) and Node 20+ (this session used Node 24.19.0)
- No OCR, no Tesseract, no Poppler — `SCOPE.md` explicitly excludes OCR/scanned PDFs (text-layer PDFs only)
- No `ANTHROPIC_API_KEY` needed at runtime — scoring is 100% deterministic keyword/regex matching, not Claude-judged
- On Windows: `python-magic` needs a native `libmagic` — `requirements.txt` resolves this with a platform marker (`python-magic-bin` on `win32`)

CDI SCORER — SOURCE CODE ORGANIZATION

Source Code Organization

backend/ Flask application-factory service

(app.py): extraction/ (PDF/DOCX/TXT),
phi/ (Presidio redaction), validation/
(rubric_loader.py,
dimension_indicators.py, matcher.py,
scorer.py),
schemas/validation_result.py
(dataclasses), routes/ (validate.py,
rubrics.py, health.py), tests/.

frontend/src/ React (Create React App,

functional components + hooks). App.jsx
orchestrates upload → result;
components/UploadForm.jsx,
ResultsSummary.jsx, CriteriaList.jsx,
GapsList.jsx, RubricSelector.jsx (active-
rubric display, no selection step);
api/client.js isolates all fetch calls.

Project root: demo/ (CLAUDE.md, SCOPE.md, SPEC.md, README.md, backend/, frontend/) —
SCOPE.md and SPEC.md were already checked into the repo and binding *before* this build session began;
CLAUDE.md was the standing project-instructions file Claude Code loaded automatically at session start.

UNIT 5 — SUMMARY OF STEPS PERFORMED IN THIS SESSION

Summary of Steps Performed in this Session

Same 9-step Define → Build → Validate flow as the reference SDK build — but **every step from 3 onward ran inside one interactive Claude Code session** (this conversation), not a scripted Agent SDK loop. Steps 0–2 (problem statement through system spec) were already resolved in the repo when the session opened.

#	Step	Who	What happened in this session
0	Problem statement	Human	Already captured in <code>SCOPE.md</code> before session start.
1	Refine problem statement	LLM	Already resolved — <code>SCOPE.md</code> 's "Ambiguous Areas" section.
2	Write system spec	LLM	Already written — <code>SPEC.md</code> , a binding 320-line contract.
3	Write system prompt for Claude Code	LLM	Already written — <code>CLAUDE.md</code> , auto-loaded as project instructions.
4	Claude Code builds it	Claude Code	This session: full backend + frontend written directly from <code>CLAUDE.md/SCOPE.md/SPEC.md</code> .
5	Run on a real document	Standalone code	Ran the scorer/matcher via a standalone script against hand-built <code>.txt</code> fixtures.
6	Self-validation	Standalone code	<code>pytest</code> — 24/24 passed, incl. 2 golden-file regression cases.
7	Human review	Human	User ran backend + frontend and tested the live UI manually.
8	Accept	Human	"I have tested manually, it is working fine. We are done with this development."

UNIT 6 — DETAILED STEPS (REAL ARTIFACTS)

STEP 0 · Problem Statement

Human

Purpose: capture the one-paragraph ask, with no implementation detail.

What actually happened: this step was already done before Claude Code opened the repo. The problem statement lives verbatim at the top of `SCOPE.md`:

SCOPE.md (real file, lines 1-4)

```
# SCOPE.md — Medical Record Validator (CDI Scorer)

## Problem Statement (as given)
Build a medical record validation tool that checks records against a
predefined rubric.
```

Why this matters: everything downstream — the rubric's weighted narrative-level structure, the extraction failure taxonomy, the PHI-handling rules, the hard-rule overrides — is derived from this single sentence plus the actual shape of the supplied `med_record_rubrics.json` file, exactly the way the reference workflow describes it.

UNIT 6 — DETAILED STEPS (REAL ARTIFACTS)

STEP 1 · Refine the Problem Statement

LLM

What actually happened: also already resolved before this session — `SCOPE.md`'s "Ambiguous Areas — Decisions Needed" section is the refinement record: nine ambiguities, each with a proposed default and, for most, a note on how `SPEC.md` resolved it. Three real examples:

Ambiguity	Resolution actually shipped
Rubric criteria structure	RESOLVED by SPEC.md §2.1: not keyword/regex criteria at all — ten fixed dimensions (R1–R10), each scored 1–5 against prose level descriptions, weighted, summed to 0–100.
Partial credit	RESOLVED: each dimension resolves to a graded 1–5 score (or N/E, "no evidence," 0 points) — not pass/fail.
Extraction failure handling	Default kept: return a clear error with no partial/fallback OCR attempt; the run does not proceed to validation.

Why this matters: Step 1's real value here was closing off the exact same category of ambiguity the reference guide calls out — rubric format/source and failure-case behavior — *before* any code existed, so Step 4 had zero open design questions to guess at.

UNIT 6 — DETAILED STEPS (REAL ARTIFACTS)

STEP 2 · Write the System Specification

LLM

What actually happened: `SPEC.md` (320 lines) was already the binding contract for this session — architecture, extraction pipeline, PHI rules, the matching algorithm, the response JSON schema, and the full API contract. Real scoring rule, quoted directly:

`SPEC.md §2.1 / §2.3 (real file, excerpt)`

```
formula: points = (score / 5) * weight    # per dimension
total:   sum of all ten dimensions, 0-100

decision_bands:
  88-100  DOCUMENTATION ACCEPTED
  74-87   ACCEPTED WITH QUERY
  58-73   RETURN FOR CLARIFICATION
  0-57    DEFICIENT

hard_rules:
  HR-1  R1 (Authentication) == 1  ->  DEFICIENT          # overrides weighted band entirely
  HR-2  R5 (Diagnostic Specificity) == 1  ->  RETURN FOR CLARIFICATION
```

Why the hard-rule override exists: a documentation-integrity rubric can't let a high weighted average paper over an unauthenticated record (R1) or a diagnosis the narrative can't support (R5) — the spec makes those two dimensions capable of overriding the weighted-band lookup entirely, and my `scorer.py` implements exactly that precedence (see Unit 7).

UNIT 6 — DETAILED STEPS (REAL ARTIFACTS)

STEP 3 · Write the System Prompt for Claude Code

LLM

What actually happened: unlike the Agent-SDK variant, there is no separately-authored `system_prompt` string — **Claude Code's system prompt equivalent is `CLAUDE.md`**, loaded automatically from the project root at the start of this session and re-applied on every turn. It carried the same load-bearing weight the reference guide's `system_prompt.md` does:

CLAUDE.md (real file, excerpt)

```
IMPORTANT: These instructions OVERRIDE any default behavior

## The rubric file is READ-ONLY to generated code
- Never write code that modifies, regenerates, or "fixes" this file.
- Load it exactly once, at app startup, in rubric_loader.py.

## PHI handling — hard constraints, not suggestions
1. Never write extracted record text to disk or any persistent store.
2. Never log raw extracted text — pass through phi/redact.py first,
   including exception tracebacks.
4. The ONLY place raw text may flow: (a) the rubric matcher, and
   (b) matched_snippet fields in the JSON response.

## Golden regression suite — must always pass
- Never edit an existing golden fixture to make a failing test pass.
```

Standing rule that made every later slice trustworthy: *"If a task seems to require changing this file's structure ... stop and ask rather than editing it"* — this is CLAUDE.md's version of the reference guide's "self-check before done" rule: a hard boundary Claude Code was not permitted to cross silently.

UNIT 6 — DETAILED STEPS (REAL ARTIFACTS)

STEP 4 · Claude Code Builds It

[Claude Code](#)

What actually happened: this is where this Claude Code session's real work is — not a scripted SDK loop, an ordinary interactive session using `Write/Edit/Bash` tools directly, slice by slice, matching the phase breakdown the reference guide uses for its own SDK runs:

1. **Backend scaffold** — `config.py`, `extraction/errors.py`, `detect.py`, per-type extractors (`pdf_extractor.py`, `docx_extractor.py`, `txt_extractor.py`).
2. **PHI module** — `phi/presidio_config.py` (custom AADHAAR/PAN regex recognizers + Presidio defaults), `phi/redact.py` (the single redaction helper).
3. **Schemas** — `schemas/validation_result.py`, dataclasses mirroring SPEC.md §3 field-for-field.
4. **Validation engine** — `rubric_loader.py` (load-once), `dimension_indicators.py` (hand-authored keyword/regex indicator sets, 10 dimensions × 5 levels, derived from the rubric's own prose), `matcher.py` (top-down level assignment), `scorer.py` (weighted aggregation + hard rules).
5. **API wiring** — `routes/health.py`, `routes/rubrics.py`, `routes/validate.py`, `app.py` (application factory, blueprint registration, CORS).
6. **Tests** — `test_extraction.py`, `test_matcher.py`, `test_golden.py` + two golden fixtures, `test_api.py`.
7. **Frontend** — `App.jsx`, `api/client.js`, all five components, styles.

One real "loop needed" moment inside this step: `requirements.txt` initially pinned `spacy==3.7.5` — that pin has no prebuilt wheel for this machine's Python 3.14, so a clean install failed compiling `blis/thinc` from source. Relaxed the pin to `spacy>=3.7.5`; a fresh venv then installed clean and all 24 tests passed. This is the same category of "self-check catches an environment gap" moment the reference guide's judged-scorer failure illustrates (Unit 6, Step 6/8) — except this one was diagnosed and fixed within Step 4, not carried forward as an accepted gap.

UNIT 6 — DETAILED STEPS (REAL ARTIFACTS)

STEP 5 · Run the Parser on a Real Document

[Standalone code](#)

What actually happened: bypassed the Flask API entirely and called `rubric_loader + scorer` directly from a standalone script against a hand-built `.txt` fixture — no model involved, since scoring here is pure keyword/regex matching, not LLM judgment.

```
python run_scorer.py record_1.txt (real captured output, excerpt)

"overall": {
  "score_points": 45.0,
  "decision": "RETURN FOR CLARIFICATION",
  "hard_rules_triggered": [{"id": "HR-2", "rubric_id": "R5",
    "action": "RETURN FOR CLARIFICATION"}]
}
"dimension_results"[0]: { "rubric_id": "R1", "score": 5, "points_earned": 8.0,
  "matched_snippet": "Electronically signed by Jane Smith, MD,..." }
"dimension_results"[4]: { "rubric_id": "R5", "score": 1, "points_earned": 2.8,
  "matched_snippet": "...Assessment: rule out pneumonia given the clinical pi...",
  "hard_rule_triggered": "HR-2" }
```

Verification, not just execution: the expected score (45.0) and decision were independently hand-computed from the rubric weights and indicator logic *before* running the script, then cross-checked against this real output — they matched exactly. A second fixture with no matching keywords anywhere (`record_2.txt`) was also run and confirmed to land on all-N/E, 0.0 points, DEFICIENT.

UNIT 6 — DETAILED STEPS (REAL ARTIFACTS)

STEP 6 · Self-Validation (Golden-File Regression)

[Standalone code](#)

What actually happened: the two hand-verified records from Step 5 became `tests/golden/record_1.txt` and `record_2.txt`, with their score-verified JSON output frozen as `*.expected.json`. The full suite (matcher, extraction, golden, API) was then run for real from a clean install:

```
pytest -v (real captured output)

tests/test_api.py::test_health PASSED
tests/test_api.py::test_rubrics PASSED
tests/test_api.py::test_validate_missing_file_returns_400 PASSED
tests/test_api.py::test_validate_unsupported_type_returns_415 PASSED
tests/test_api.py::test_validate_empty_content_returns_422 PASSED
tests/test_api.py::test_validate_success_returns_full_schema PASSED
tests/test_api.py::test_validate_file_too_large_returns_413 PASSED
tests/test_extraction.py (8 tests) PASSED
tests/test_golden.py::test_golden_record[record_1...] PASSED
tests/test_golden.py::test_golden_record[record_2...] PASSED
tests/test_matcher.py (5 tests) PASSED

===== 24 passed in 1.31s =====
```

Reading a real environment gap this step caught: the first run of `test_api.py` segfaulted on `import magic` — Windows has no native `libmagic`. Root cause traced (not a code defect), fixed with a `sys_platform` marker in `requirements.txt` (python-magic-bin on win32). Re-run from a **fresh venv**, clean install, all 24 pass. This is the same "self-check surfaces an environment gap, not a code defect" pattern the reference guide's missing-API-key finding illustrates.

UNIT 6 — DETAILED STEPS (REAL ARTIFACTS)

STEP 7 · Human Review

Human

What actually happened — honestly, not idealized: Claude Code started both servers and confirmed liveness itself, but did **not** personally view the rendered UI. An attempt to drive the browser automation tool against `localhost:3000/5000` failed — the tool's Chrome instance could not reach either port even though both were confirmed live via `curl`, indicating a network-namespace split between the shell and the browser tool, not an app defect. Rather than guess further, Claude Code stopped and handed the live app to the user for a real manual check.

```
curl http://127.0.0.1:5000/api/v1/health (real captured output)
```

```
{"status":"ok"}  
HTTP_STATUS:200
```

```
npm run build (real captured output, frontend)
```

```
Compiled successfully.  
  
File sizes after gzip:  
47.08 kB   build/static/js/main.61d0cf5a.js  
784 B     build/static/css/main.2d9c11df.css
```

What the user actually confirmed: *"I have tested manually, it is working fine."* — the real, human-driven end-to-end check (upload → extract → score → report, rendered in a real browser) that Claude Code's own tooling could not perform in this environment.

UNIT 6 — DETAILED STEPS (REAL ARTIFACTS)

STEP 8 · Accept

Human

Accepting a slice means: its self-check passes for real, and human review against the running app didn't surface a behavior the self-check couldn't see.

Slice	Self-check	Human review	Disposition
Extraction (PDF/DOCX/TXT)	PASS (8 tests)	Not independently re-verified live	Accepted
Matcher / rubric scoring	PASS (5 tests + 2 golden)	Not independently re-verified live	Accepted
API contract	PASS (7 tests)	User confirmed via live UI	Accepted
Full UI flow (upload → report)	N/A (no self-check covers rendering)	PASS — user's own manual test	Accepted
Claude Code's own browser verification	—	Blocked: browser tool network-isolated from dev servers	Loop → handed to human (Step 7)

Final acceptance, quoted directly:

"I have tested manually, it is working fine. We are done with this development."

UNIT 7 — UNDERSTAND THE PROJECT STRUCTURE AND CODE

Directory layout, from `CLAUDE.md` (the real, load-bearing structure Claude Code built against — reproduced exactly, no restructuring permitted):

```
demo/
  CLAUDE.md  SCOPE.md  SPEC.md  README.md
  backend/
    app.py  config.py  requirements.txt  med_record_rubrics.json
    routes/
      validate.py  rubrics.py  health.py
    extraction/
      detect.py  pdf_extractor.py  docx_extractor.py  txt_extractor.py  errors.py
    phi/
      presidio_config.py  redact.py
    validation/
      rubric_loader.py  dimension_indicators.py  matcher.py  scorer.py
    schemas/
      validation_result.py
    tests/
      golden/  test_golden.py  test_extraction.py  test_matcher.py  test_api.py
  frontend/src/
    App.jsx  api/client.js
    components/  UploadForm.jsx  RubricSelector.jsx  ResultsSummary.jsx  CriteriaList.jsx
    GapsList.jsx
```

Every response field is a dataclass, not a bare dict crossing a function boundary. Real excerpt from `backend/schemas/validation_result.py`:

```
@dataclass
class DimensionResult:
    rubric_id: str
    name: str
    weight: int
    score: DimensionScore          # int 1-5, or literal "N/E"
    points_earned: float
    matched_level_text: str
    matched_snippet: str | None
    hard_rule_triggered: str | None
```

API contract (base path `/api/v1`, no auth, no persistence — matching `SPEC.md` §4 exactly):

Method & path	Purpose	Success
POST /validate	upload + extract + score, synchronous	200, schema in <code>SPEC.md</code> §3
GET /rubrics	metadata for the single active rubric	200
GET /health	liveness	200 {"status":"ok"}

Coding conventions actually enforced: type hints on every signature; specific exception classes (`ExtractionError` subtypes), never bare `except :`; PHI redaction isolated to `phi/redact.py`, imported

everywhere else rather than reimplemented; rubric content always read from `med_record_rubrics.json`, loaded exactly once at startup, never re-read per request, never written to.

UNIT 8 — STEP-BY-STEP PROCEDURE TO REBUILD

Step 1: Set Up the Backend Environment

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

```
python app.py (real captured behavior)
```

```
* Running on http://127.0.0.1:5000
GET /api/v1/health -> 200 {"status":"ok"}
```

Step 2: Run the Test Suite

```
pytest
```

Real result on this build: **24 / 24 tests passed** from a clean install — no environment caveats remained once the `spacy` pin and Windows `libmagic` marker were fixed (Unit 6, Steps 4/6).

Step 3: Run the Frontend

```
cd frontend
npm install
npm start
```

Vite/CRA dev server on `http://localhost:3000`, proxying API calls to `http://localhost:5000/api/v1` per `frontend/.env`.

Step 4: Verify End to End

1. Upload a PDF/TXT/DOCX record through the browser and confirm a scored report renders.
2. `npm run build` — real result: **compiled successfully**, 47.08 kB gzipped JS bundle.